CSE438 • PROJECT 12 • TECHNICAL EXPLANATORY DOCUMENT

FractureFuseNet: Phase 1 Summary & Architectural Changes Explanation

Topic: Multimodal Late Fusion of Deep Features & Classical Edge/Hough Line Detection

Scope: Phase 1 Preprocessing Audit, Architectural Rationale & Design Modifications | **Date:** November 2026

PART 1: Brief Summary of All Phase 1 Work

In Phase 1, we audited, debugged, and standardized the raw *Human Bone Fractures Multi-modal Image Dataset* (HBFMID) into a mathematically validated, leakage-free PyTorch multimodal pipeline:

1. **Dataset Integrity Audit:** Audited **1,539 images** (640 × 640 RGB) and **1,539 YOLO annotations** (1,771 bounding boxes). Confirmed 0 corrupt images, uniform dimensions, and 0 invalid bounding box coordinates.
2. **Annotation Cleaning & Disambiguation:** Evaluated all multi-box images. Preserved **1,429 clean single-label images** (1,423 unanimous + 6 strict majority). Safely excluded **110 ambiguous records** (107 multi-class ties + 3 empty label files) with complete audit trail logging in `excluded_records_audit_log.csv`.
3. **Eliminating Roboflow's 40.8% Data Leakage:** Discovered Roboflow's original split leaked augmented copies across train and test (40.82% base ID leakage, 12.79% perceptual duplicate leakage). Solved this using **dihedral perceptual hashing (`pHash` + `dHash` ≤ 10)**, discovering **657 true source clusters** (matching the ~641 raw radiographs reported by Mendeley).
4. **Group-Aware Stratified Splitting (60:20:20):** Implemented 5-fold `StratifiedGroupKFold` on the 1,429 clean images grouped by composite `group_id`. Achieved **0.00% cross-split leakage** (Train: 857, Valid: 285, Test: 287).
5. **Pigeonhole Principle Proof on Rare Class (`Segmental`):** Proved that `Segmental` has only 8 clean images from 2 source groups (`70_jpg` and `225_jpg`). Mathematically, $n=2$ groups cannot populate $k=3$ splits without leakage. Placed them in Train (5) and Valid (3) with 0 in the fixed Test split, while evaluating 100% of them out-of-fold across 5-fold cross-validation.
6. **Modality Confounding Audit (X-ray vs MRI):** Discovered 64.6% of `Comminuted` fractures are MRI scans, while 5 classes have 0% MRI. Established **X-ray ($N=1,227$) as the primary benchmark** to prevent the model from learning "MRI texture = fracture" shortcuts.
7. **Classical 64-Dimensional Feature Extraction:** Extracted 64 structural features from full-resolution radiographs via CLAHE, multi-threshold Canny edge detection, Hough Transform lines, circular orientation statistics ($2 heta$), and Sobel gradients.
8. **Cluster-Level Statistical Hypothesis Testing:** Corrected pseudoreplication (which inflated naive p-values by up to $1.5 imes 10^7 imes$). Conducted 10,000 permutation tests on $N=496$ independent clusters with Holm and Benjamini-Hochberg FDR adjustments. Proved classical features capture macro-anatomical structure (Cliff's $\delta = -0.698$, $p < 10^{-4}$).
9. **PyTorch Multimodal Pipeline & Automation:** Built a PyTorch DataLoader pipeline with shared geometric transform symmetry, train-only `StandardScaler` provenance, Focal Loss validation, 16/16 unit tests, and a 12/12-check master pipeline runner (`run_phase1_pipeline.py`) executing in 14.2s.

PART 2: Detailed Architectural Changes & Scientific Explanations

1. Why convert the image into 3 × 224 × 224? What is "3" here?

In PyTorch, 2D convolutional layers represent images as 3-dimensional tensors:

> *$$ ext{Tensor Shape} = (ext{Channels}, ext{Height}, ext{Width}) = (C, H, W)$$*

A. What is "3" here?

- **`3` represents the 3 color channels: Red, Green, and Blue (RGB).**
- *The core question:* Medical X-rays and MRI scans are monochrome (grayscale) images. Why do we encode them into 3 channels instead of 1?
- **The Reason (Pretrained Transfer Learning):** Standard state-of-the-art vision backbones (e.g., **ResNet-50, EfficientNet-B0, DenseNet-121, ConvNeXt**) were pretrained on the ImageNet dataset (1.2 million natural color photos). The very first convolutional layer (`conv1`) of these networks strictly expects an input with **`in_channels = 3`** :

```
nn.Conv2d(in_channels=3, out_channels=64, kernel_size=7, stride=2, padding=3, bias=False)
```

- If we fed a 1-channel grayscale tensor `(1, 224, 224)` :
  - PyTorch throws a shape mismatch runtime error: "*Expected 3 channels, got 1*".

    - To bypass the error, you would have to replace the first layer with `in_channels=1` and randomly re-initialize its weights. Doing that **destroys all pretrained ImageNet low-level feature detectors** (Gabor edge, texture, and curve filters) learned by the network!
- **Our Solution:** We broadcast the grayscale radiograph across all three channels ($R = G = B$). This allows the pretrained CNN to immediately utilize its rich pretrained feature representations without architectural alterations or loss of transfer learning capability.

B. Why 224 × 224?

The raw images in the dataset are $640 imes 640$ pixels. Resizing to $224 imes 224$ serves three critical purposes:

1. **Standard Architecture Resolution:** $224 imes 224$ is the canonical input resolution for which ResNet, DenseNet, and EfficientNet architectures were designed and benchmarked.
2. **Computational & Memory Efficiency:**
    - A $640 imes 640$ image contains **409,600 pixels**.
    - A $224 imes 224$ image contains **50,176 pixels** — an **8.16× reduction in memory and FLOPs**!
    - Training deep CNNs on $640 imes 640$ exhausts GPU VRAM, forcing tiny batch sizes (e.g. 2 or 4) that destabilize Batch Normalization. At $224 imes 224$, standard batch sizes of 16, 32, or 64 run smoothly.
3. **The Core Innovation of FractureFuseNet (Dual-Branch Resolution Strategy):**
    - *Won't downsampling from 640 to 224 blur thin hairline fractures?* Yes! That is **the exact reason standard CNNs struggle with fine fractures**.
    - In **FractureFuseNet**, we resolve this through our dual-branch design:
        - **Branch A (Deep CNN):** Receives the resized **$224 imes 224$** image to capture *global anatomical context* (e.g. recognizing whether the bone is a femur, ulna, radius, or tibia).
        - **Branch B (Classical Edge/Hough):** Operates on the **native full-resolution $640 imes 640$ image**! It detects sharp linear discontinuities, cortical steps, and edge gradients at sub-millimeter pixel precision.
    - Both representations are fused at the classifier head (Late Fusion). This gives both global contextual understanding and full-resolution fracture edge precision.

2. What Other Changes Were Made and Why?

A. Shared Geometric Augmentation (`apply_shared_geometric_transform`)

- **What was changed:** If a training radiograph undergoes random horizontal flip ($p=0.5$) or random rotation ($\pm 15^\circ$), the **exact same geometric transformation** is applied to both Branch A and Branch B simultaneously.
- **Why:** In naive pipelines where PyTorch transforms run independently, Branch A would receive a flipped radiograph (bone pointing left) while Branch B would extract classical features from an unflipped radiograph (bone pointing right). This spatial discrepancy creates an irreconcilable conflict when the fusion layer concatenates the features. Our shared transform preserves strict spatial alignment.

B. Removing Brightness and Contrast Shortcuts from Classical Features

- **What was changed:** We removed raw image mean intensity and standard deviation from the 64-dimensional feature vector, replacing them with **axial circular orientation statistics ($2 heta$ doubling)** and **spatial edge heterogeneity**.
- **Why:** In exploratory data analysis, we found MRI scans were systematically darker than X-rays, and different hospital scanners produced varying exposures. If raw brightness is present in the feature vector, machine learning models learn a trivial shortcut (e.g., classifying an image as *Comminuted* purely because it is dark, rather than recognizing bone fragmentation). Forcing CLAHE contrast equalization and pure geometric descriptors ensures the classical branch learns genuine fracture features.

C. Solving Cross-Split Leakage via Dihedral Perceptual Hashing

- **What was changed:** Replaced naive filename splitting (`filename.split('.rf.')[0]`) with **dihedral `pHash` + `dHash` clustering at Hamming distance ≤ 10 bits**.
- **Why:**
  1. Roboflow flattened separate class folders numbered `1.jpg..225.jpg` into a single directory. Consequently, unrelated radiographs from different classes shared the same prefix (`225_jpg` had both Segmental and Oblique Displaced images from different patients).
  2. Roboflow's random partition placed augmented variants (flips, rotations) of the exact same bone in both train and test (**40.82% leakage**). Evaluating on training duplicates produces false, inflated test accuracy. Our composite grouping guaranteed **0.00% leakage**.

D. Train-Only Parameter Fitting with Cryptographic Provenance

- **What was changed:** Channel normalization statistics (`mean = [0.2555, 0.2555, 0.2555]` , `std = [0.2594, 0.2594, 0.2596]`) and the 64-dimensional `StandardScaler` are fitted **strictly on the 857 training images**. Cryptographic SHA-256 provenance hashes are saved in `edge_feature_scaler_provenance.json` .
- **Why:** Fitting scalers or computing normalization across the whole dataset (Train + Valid + Test) constitutes **Data Snooping / Lookahead Bias**. Test data must remain completely unseen.

E. Correcting Pseudoreplication in Statistical Hypothesis Testing

- **What was changed:** In `verify_cv_hypothesis.py` , instead of running tests on all 1,227 augmented X-ray images, features were aggregated to their **496 independent source clusters** for 10,000 permutation tests with Holm and Benjamini-Hochberg FDR corrections.
- **Why:** Testing on augmented copies violates the independent observation assumption (pseudoreplication), artificially inflating statistical significance by up to **15 million times** ($p = 6.89 imes 10^{-17}$ vs true cluster $p = 1.06 imes 10^{-9}$). Aggregating to cluster medians ensures scientific validity.

F. Integrating Class-Weighted Focal Loss ($\gamma = 2.0$)

- **What was changed:** Implemented `FocalLoss(alpha=class_weights, gamma=2.0)` to supersede standard Cross-Entropy.
- **Why:** The dataset has severe class imbalance (`Transverse Displaced` has 568 images vs `Linear` with 22 and `Segmental` with 8). Standard Cross-Entropy allows easy majority-class samples to dominate gradient updates; Focal Loss down-weights easy examples and forces the model to focus on difficult minority fractures.

PART 3: Architectural Comparison & Summary Table

Component	Previous / Naive Approach	Corrected Phase 1 Implementation	Scientific / Engineering Rationale
Deep Input Shape	Variable / unaligned	**`torch.float32 [Batch, 3, 224, 224]`**	Allows instant plug-and-play with pretrained ImageNet CNN backbones without re-initializing layer 1 weights.
Classical Input Shape	Resized / distorted	**Native (640 × 640) Grayscale + CLAHE**	Preserves high-frequency cortical fracture lines and hairline cracks that CNN downsampling blurs.
Data Augmentation	Independent random transforms	**`apply_shared_geometric_transform`**	Guarantees exact geometric and spatial alignment between the deep stream and classical stream.
Duplicate Grouping	Filename prefix `base_id`	**Dihedral `pHash` + `dHash` ≤ 10 + Union-Find**	Eliminates 40.82% cross-split leakage and clusters 657 true source radiographs.
Scaler Fitting	Fit on whole dataset	**Train-Split Only ($N=857$) + SHA-256 Provenance**	Eliminates data snooping / lookahead bias into validation and test sets.
Statistical Testing	Image-level testing ($N=1,227$)	**Cluster-level testing ($N=496$) + 10k Permutations**	Eliminates pseudoreplication and Type I error inflation.
Loss Function	Standard Cross-Entropy	**Class-Weighted Focal Loss ($\gamma=2.0$)**	Prevents majority classes (Transverse Displaced) from dominating rare classes (Linear, Segmental).